PrivacyGuard Solution Design Document

Nov 6, 2026 · Author: Thangadurai Thambiannan

1. Document control

This document describes the design of the PrivacyGuard managed package in release 0.5.0. It's written for Salesforce architects, developers, administrators and security reviewers. Every component named here exists in the `force-app` source.

Item	Value
Product	PrivacyGuard
Publisher	JM Fortune Pty Ltd
Package type	Second-generation managed package (2GP)
Namespace	`privacyguard`
Package ID	0HoQE0000000GUb0AM
Version / build	0.5.0
Subscriber package version	04tQE00000jpLenYAE
Apex API version	66.0
Lightning app	Data Masking Framework (labelled PrivacyGuard)
Contact	privacyguard@sandboxshield.com.au

Source inventory: 49 Apex classes, 49 Apex test classes, 4 Apex triggers, 9 Lightning Web Components, 9 custom objects, 3 hierarchy custom settings, 4 custom metadata types (39 packaged records), 6 custom permissions, 7 permission sets, 10 tabs, 1 Named Credential, 1 External Credential and 1 External Auth Identity Provider.

Revision history

Revision	Date	Source reviewed	Changes
1.2	9 Nov 2026	Version 0.5.0 (04tQE00000jpLenYAE)	Masking now enforces object and field edit access for the running user while still bypassing record sharing; recovery batch query made explicit SYSTEM_MODE with bind variables; PII scan results inserted in user mode; MaskingConfigPromoter uses WITH USER_MODE; field-configuration lookup filtered by requested objects; keyword override update moved out of its loop. Security-scan remediation: explicit `as system` writes in the trigger-status job, explicit `SYSTEM_MODE` recovery log query, batched logging, a single bulk run-object update at launch, `LIMIT` clauses on configuration queries, and new test `PrivacyGuardMaskingFlsTest`. Sections 1, 5, 7, 8, 11 and 14 updated.
1.1	9 Nov 2026	Version 0.3.0 (04tQE00000jXzkLYAS)	Object-wide file purge retired; production-override components neutralised and relabelled; all PrivacyGuard objects set to Private; failure-log text suppressed and audit log made immutable (new trigger `PGProtectMaskingLog`); Admin log access made read-only. Sections 1–2, 4, 5, 7, 8, 10, 11, 13 and 14 updated.
1.0	6 Nov 2026	Version 0.2.0, build 0.2.0.3 (04tQE00000jUHmHYAW)	First issue

2. Business context and scope

PrivacyGuard lets Salesforce administrators make sandbox copies of production data safe for development, testing and training. It finds fields likely to hold personal data and replaces their values with realistic or blank substitutes, and it does all processing inside the subscriber org.

Objectives

- Find candidate PII fields from field type, a keyword dictionary and Salesforce Data Classification.
- Let an administrator approve each field and choose how it is masked, before any data changes.
- Run repeatable, scheduled masking jobs with per-record completion tracking and failed-record recovery.
- Keep subscriber automation out of the way during a run, either through cooperative bypass flags or by temporarily deactivating eligible Apex triggers.
- Let administrators review and remove selected legacy Attachments and Salesforce Files from sandbox data.
- Record every configuration change and operation for review.

In scope

Capability	Summary	Org restriction
PII discovery	Scans object fields and stores candidate findings for review	Any org (no record data read or changed)
Masking configuration	Field-level rules using 10 built-in strategies or a custom Apex class	Any org
Masking jobs	Reusable jobs over one or more objects, run serially or in parallel, manually or on a schedule	Execution: sandbox only
Preview	Shows before and after values for up to 5 records without saving	Any org; user-mode read
Failed-record recovery	Re-runs only the records that failed in a previous run	Sandbox only
Apex trigger management	Deactivates and restores eligible unmanaged Apex triggers through the Tooling API	Sandbox only

Capability	Summary	Org restriction
File & attachment management	Inventory, CSV export, per-object counts and selected-item deletion	Deletion: sandbox only
Audit Centre and dashboard	Activity history, run details, configuration coverage	Any org

Out of scope

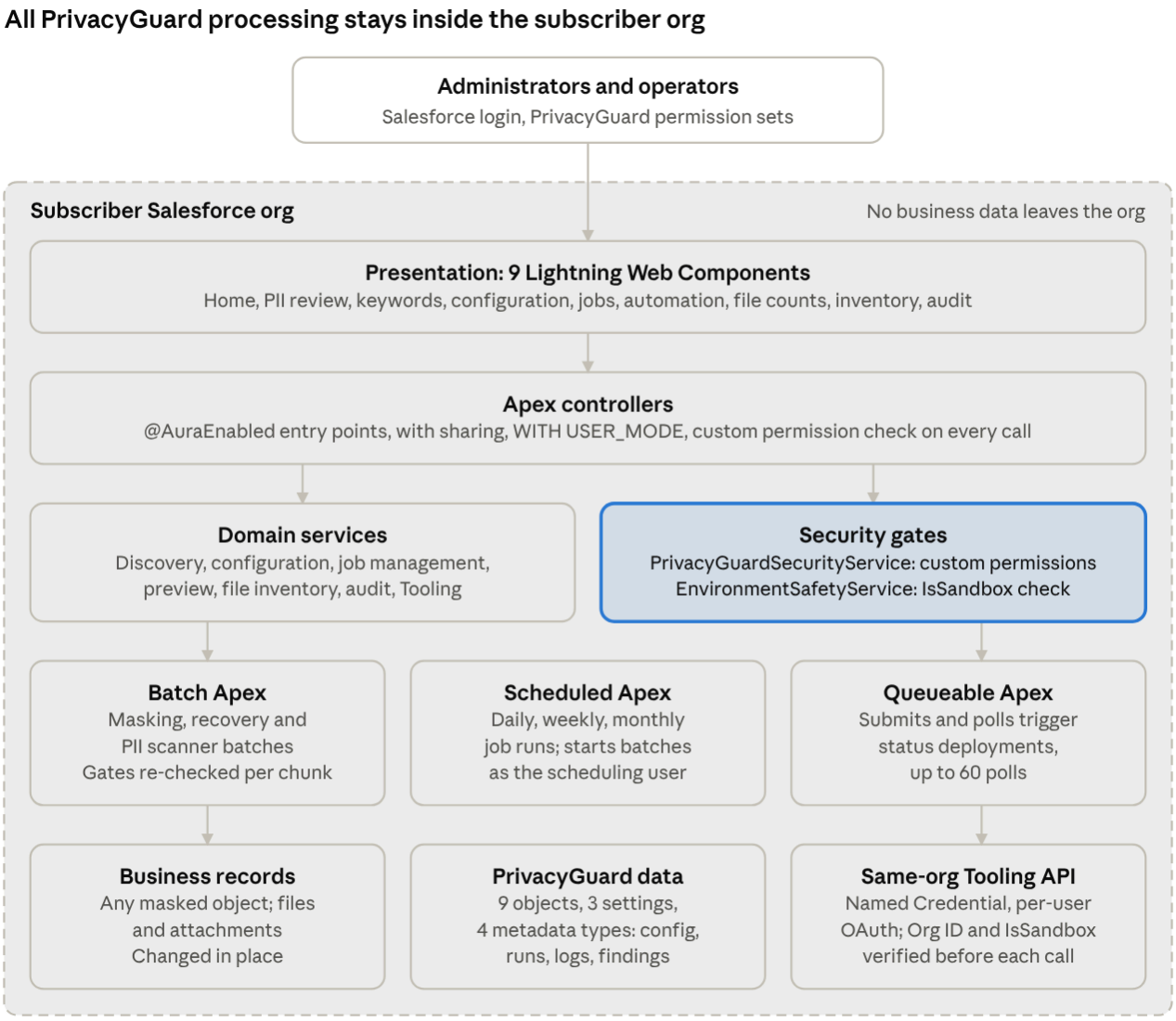
- Masking or deleting data in production orgs. There is no override.
- Flow and Validation Rule metadata toggling. This was retired, and the remaining entry points throw an error.
- Delta (timestamp-based) masking. Retired in favour of the completion checkbox.
- Generating or seeding test data.
- Object-wide file and attachment purge. Retired in the remediation release; remaining entry points fail closed.
- Classifying the contents of files.
- Reversing masking or deletion.

Assumptions

- The package is installed in a sandbox created from production. Discovery and configuration may also be done in production.
- Each object to be masked has an editable `Data_Masking_Complete__c` checkbox that defaults to false. The package supplies it on Account, Contact and Lead.
- Subscriber automation that should pause during masking is written to read the `Automation_Bypass__c` flags, or is an unmanaged Apex trigger that can be deactivated.
- Administrators keep a recovery option, such as a sandbox refresh, because masking can't be undone.

3. Solution architecture

PrivacyGuard is a native Salesforce application with no external services. Every request passes through permission and environment gates before any data or metadata changes. Work that changes data runs asynchronously so it can scale within governor limits.



PrivacyGuard logical architecture · 5 layers, 1 trust boundary

The only HTTP traffic is from the subscriber org back to its own My Domain, used to change Apex trigger status. The permission and sandbox gates are checked at each controller entry point and again inside every batch chunk and Tooling call.

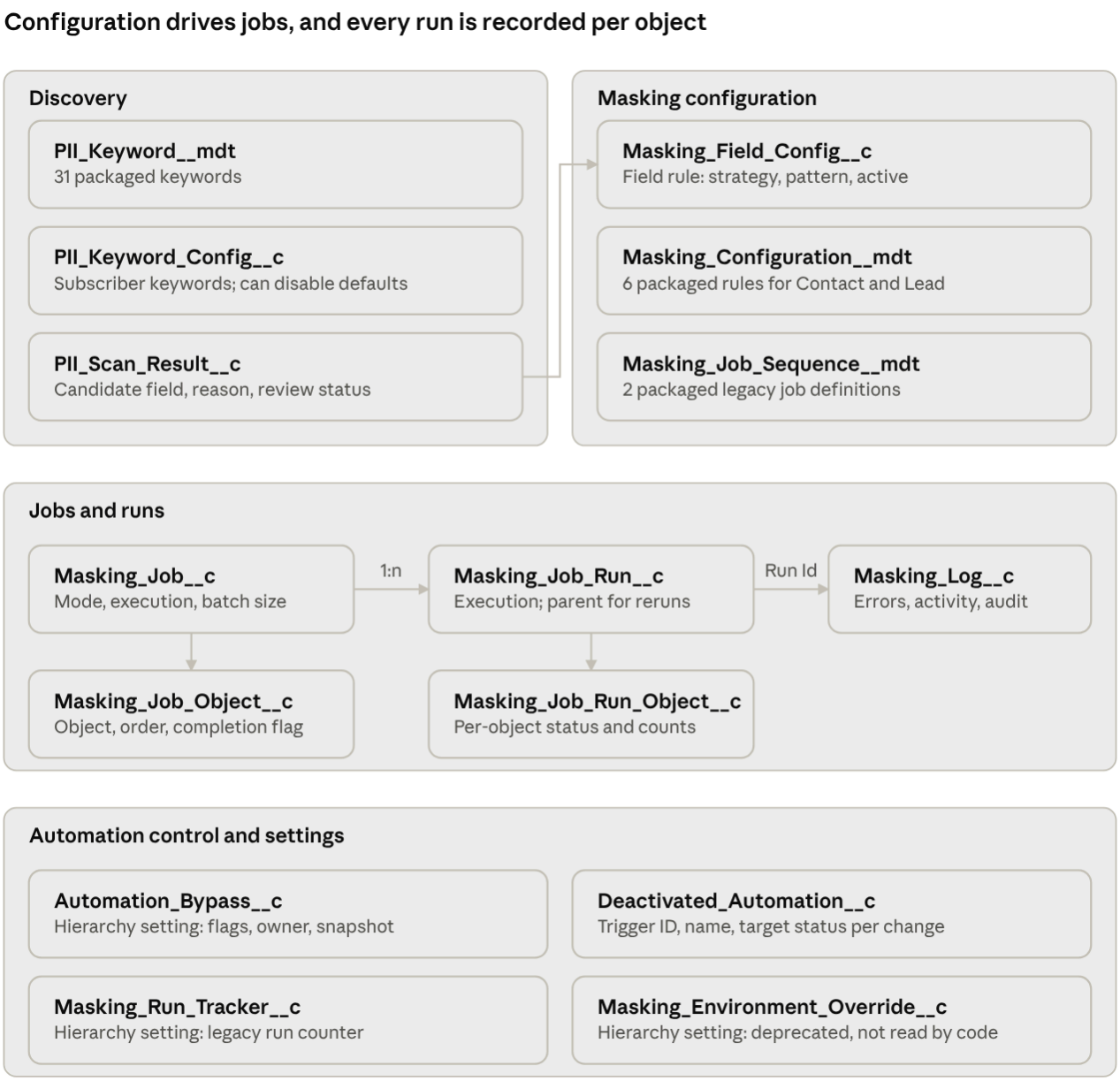
Architecture principles

Principle	How the design applies it
Platform-native	LWC + Apex + Salesforce storage; no publisher-hosted backend, no external callouts
Explicit approval before change	Discovery only suggests. Accepted findings become inactive configuration until an admin chooses a strategy and activates it.

Principle	How the design applies it
Fail closed	Unknown org type is treated as production. A record with any field that fails to mask is not saved or marked complete.
Bounded transactions	Batch Apex per object, configurable batch size (default 200), queueable polling for deployments
Persistent state	Runs, run objects, bypass snapshots and trigger operations are stored, so they can be monitored, reconciled and restored
Least privilege	Six custom permissions split discovery, masking, purge, metadata deactivation and audit. Tooling access is per user.

4. Data model

PrivacyGuard stores configuration, run history and operational state. It never stores business field values. Lookups connect jobs to their runs. Logs and trigger-operation records refer to runs by a text Run ID, so history survives if a job is deleted.



PrivacyGuard data model · 9 objects, 3 settings, 4 metadata types

Accepting a scan result creates an inactive field rule (the arrow). Scan results and field rules are matched by object and field API name, not by lookup. Field rules apply to every job that includes their object.

Object catalogue

Object	Type	Purpose	Key fields	Sharing default
`PII_Scan_Result__c`	Custom object	Review queue of candidate PII fields	Object/Field API name, Detection_Reason__c, Native_Compliance_Tag__c, Review_Status__c, Scan_Batch_Id__c	Private

Object	Type	Purpose	Key fields	Sharing default
`PII_Keyword_Config__c`	Custom object	Subscriber keywords that extend or switch off packaged keywords	Keyword__c, Is_Active__c, Notes__c	Private
`Masking_Field_Config__c`	Custom object	Per-field masking rule, created inactive	Object/Field API name, Masking_Strategy__c, Masking_Pattern__c, Execution_Group__c, Is_Active__c, Configuration_Source__c	Private
`Masking_Job__c`	Custom object	Reusable job definition	Job_Name__c, Run_Mode__c (Full / Once), Execution_Mode__c (Serial / Parallel), Batch_Size__c, Last_Status__c	Private
`Masking_Job_Object__c`	Custom object (lookup to Job)	Object included in a job, with its order	Object_API_Name__c, Sequence__c, Delta_Tracking_Field__c (holds the completion checkbox name), Is_Active__c	Private
`Masking_Job_Run__c`	Custom object (lookup to Job; self-lookup Parent_Run__c)	One execution or rerun	Run_Type__c, Status__c, Started_By__c, object and record counts, Summary__c	Private
`Masking_Job_Run_Object__c`	Custom object (lookup to Run)	Per-object progress in a run	Status__c, Async_Job_Id__c, Processed/Failed_Records__c, Error_Summary__c	Private
`Masking_Log__c`	Custom object	Errors, activity and configuration audit. Insert-only: a trigger blocks edits and deletes, and replaces failure text with a generic message	Run_Id__c, Activity_Category__c, Status__c, Error_Message__c, Technical_Details__c, Change_Details__c	Private
`Deactivated_Automation__c`	Custom object	Trigger status change tracking	Component_Id__c, Component_Name__c, Original_State_Json__c, Deployment_Status__c, Deployment_Target__c, Reactivated__c	Private
`Automation_Bypass__c`	Hierarchy custom setting (Public)	Cooperative bypass flags read by subscriber automation	Bypass_All_Automations__c, Bypass_Triggers__c, Bypass_Flows__c, Bypass_Validation_Rules__c, Bypass_Owner__c, Bypass_Snapshot__c	n/a
`Masking_Run_Tracker__c`	Hierarchy custom setting	Completion counter for the legacy metadata-driven orchestrator	Run_In_Progress__c, Total_Jobs__c, Completed_Jobs__c	n/a

Object	Type	Purpose	Key fields	Sharing default
`Masking_Environment_Override__c`	Hierarchy custom setting	Retired compatibility setting, relabelled "PrivacyGuard Retired Setting"; ignored by all code	Allow_Run_In_Production__c	n/a
`PII_Keyword__mdt`	Custom metadata (31 records)	Packaged keyword dictionary	Keyword__c	n/a
`Masking_Configuration__mdt`	Custom metadata (6 records)	Packaged sample rules, shipped inactive: Contact Email, LastName, Phone; Lead Company, Email, Phone	Object/Field API name, Field_Type__c, Masking_Pattern__c, Masking_Strategy_Class__c	n/a
`Masking_Job_Sequence__mdt`	Custom metadata (2 records)	Legacy job definitions for the metadata-driven orchestrator (Contact_Job, Lead_Job)	Job_Type__c, Run_Mode__c, Execution_Mode__c, Sequence_Order__c, Batch_Size__c	n/a

Fields added to standard objects: `Data_Masking_Complete__c` (Checkbox) on Account, Contact and Lead, plus `Data_Masking_Applied_Date__c` (Date/Time) on Contact. For any other object, the subscriber creates an editable checkbox before masking it.

5. Functional design

The app has eight functional modules, each with its own tab, LWC and controller. The table maps them. The subsections give the rules each module follows.

Module	Tab / LWC	Apex controller	Custom permission
Dashboard	PrivacyGuard Home / `privacyGuardHome`	`PrivacyGuardHomeController`	Manage PII
PII discovery and review	PII Scan Review / `piiScanReview`	`PIIScanReviewController`	Manage PII
PII keywords	PII Keyword Configuration / `piiKeywordConfiguration`	`PrivacyGuardConfigurationController`	Manage PII
Masking configuration	Data Masking Configuration / `dataMaskingConfiguration`	`PrivacyGuardConfigurationController`	Manage PII
Masking jobs, preview, runs	Masking Control Panel / `maskingControlPanel`	`MaskingControlPanelController` → `MaskingJobManagerService`	Run Masking

Module	Tab / LWC	Apex controller	Custom permission
Apex trigger management	Automation Management / `automationManagement`	`AutomationManagementController`	Metadata Deactivation
Files and attachments	Attachment File Purge / `attachmentFilePurge`, `privacyGuardFileInventory`	`FileAttachmentPurgeController`, `PrivacyGuardFileInventoryController`	Purge Files (inventory: Manage PII or Purge Files)
Audit Centre	`privacyGuardAuditCentre`	`PrivacyGuardAuditController`	View Audit

5.1 PII discovery

- **Scope:** every object that is queryable, updateable and not a custom setting. The scan runs as `PIIFieldScannerBatch` with one object per batch execution.
- **Detection rules, in order:**
  1. Field type is Email or Phone, or the field is encrypted.
  2. Keyword match. Labels and API names are normalised (camelCase split, punctuation removed) and matched on whole tokens only, so a short keyword like TIN doesn't match `OperatingHoursId`.
  3. Salesforce Data Classification. `ComplianceGroup` and `SecurityClassification` are read from `FieldDefinition`. If these can't be read, a warning is logged and the scan continues.
- **Keywords:** 31 packaged `PII_Keyword__mdt` records, merged with subscriber `PII_Keyword_Config__c` records, which can add terms or switch off defaults.
- **Exclusions:** fields the user can't read, and fields that already have a masking rule (active or not).
- **Output:** `PII_Scan_Result__c` rows with status New and the detection reason (up to 255 characters).
- **Review:** the admin accepts or removes each candidate. Accepting creates an inactive `Masking_Field_Config__c` row. The admin must then choose a strategy and activate it. `MaskingConfigPromoter` can also generate custom metadata XML for accepted rows.

5.2 Masking strategies

Strategy key	Behaviour	Pattern format
`EMAIL`	`user` + random 8 digits, keeping the original domain	None
`EMAIL_INVALID`	`sandbox.<12-character hash>@invalid.com`	None
`PHONE`	Replaces each digit with a random digit; keeps formatting	None
`NAME`	Random name from a fixed pool of 10	None
`LIST`	Random value from an admin list	`A|B|C`, up to 255 characters, no empty entries
`RANDOM_STRING`	Random alphanumeric string	Length (default 8)
`STATIC`	Fixed value, converted to the field's type	Literal value
`NUMBER_RANGE`	Random number in a range	`min|max|decimal places`
`NULL`	Clears the field (not allowed on required fields)	None
`REGEX`	Regular-expression replace	`expression=>replacement`

Null values stay null. Randomness uses `Crypto.getRandomInteger`. A rule can name a strategy class instead of a key, but `IMaskingStrategy` isn't `global`, so only classes inside the package can be used.

5.3 Masking jobs and runs

- **Job definition:** name, description, objects in order, run mode, execution mode and batch size (1–2,000, default 200).
- **Run modes:**
  - **Full** masks every record.
  - **Unmasked records only** (stored as `Once`) selects records where the completion checkbox is false or null.
  - Delta mode is rejected.
- **Execution modes:**
  - **Serial** runs objects in sequence order. A failed or partial object skips the rest.

    - **Parallel** starts one batch per object.
- **Preflight before a run starts:**
    - job is active and no run is already in progress
    - each object has active field rules
    - each object has an editable completion checkbox
    - every target field is valid for its strategy
    - the completion checkbox is not itself a masking target
- **Completion:** a record is saved, and its checkbox set to true, only when every configured field masked successfully. Otherwise the record is skipped and logged. A field is written only if the running user (for scheduled runs, the scheduling user) can edit the object, the field and the completion checkbox.
- **Scheduling:** daily, weekly or monthly (days 1–28 or last day), in the scheduling user's time zone. Scheduled runs are sandbox-only.
- **Cancellation:** aborts outstanding batch work and marks the run Cancelled. Chunks already committed stay masked.

5.4 Preview

Preview shows up to 5 records that would be masked, with before and after values for up to 100 configured fields. It needs the Run Masking permission and read access to the object, every field and the completion checkbox. The query runs `WITH USER_MODE`, and nothing is saved.

5.5 Failed-record recovery

A rerun is allowed only for a run with status Failed or Completed with Errors. `MaskingFailedRecordRecoveryBatch` reads the failed record IDs from that run's logs and re-masks only those records. The rerun is stored as a child run (`Run_Type__c = Rerun`, `Parent_Run__c`).

5.6 Apex trigger management

The module lists up to 50,000 Apex triggers. Namespaced triggers and PrivacyGuard's own three audit triggers are read-only. Only Active triggers can be deactivated, and only triggers PrivacyGuard deactivated can be restored. Each change becomes a tracked `Deactivated_Automation__c` row and a queueable deployment. Its final state is checked against the trigger's actual status. Changes stuck in "Check required" can be re-checked from the UI.

5.7 Files and attachments

Function	Rules
All files: inventory	Pages of 100 Salesforce Files and Attachments visible to the user, with links, libraries and owner. Exportable as CSV.
All files: delete selected	1–100 items per request, typed confirmation DELETE, deleted in user mode (`AccessLevel.USER_MODE`), audited. The only supported deletion path.
By object: counts	Counts Attachments and file links for 1–10 eligible objects, as seen by the user. Read-only.
By object: purge (retired)	Disabled. The LWC button is permanently disabled and labelled "Object-wide purge disabled". `FileAttachmentPurgeService.purgeForObjects` and `FileAttachmentDeletionBatch` check the sandbox gate and the Purge permission, then always throw `ObjectPurgeDisabledException`. No records are queried or deleted, so already-scheduled or direct calls fail closed.

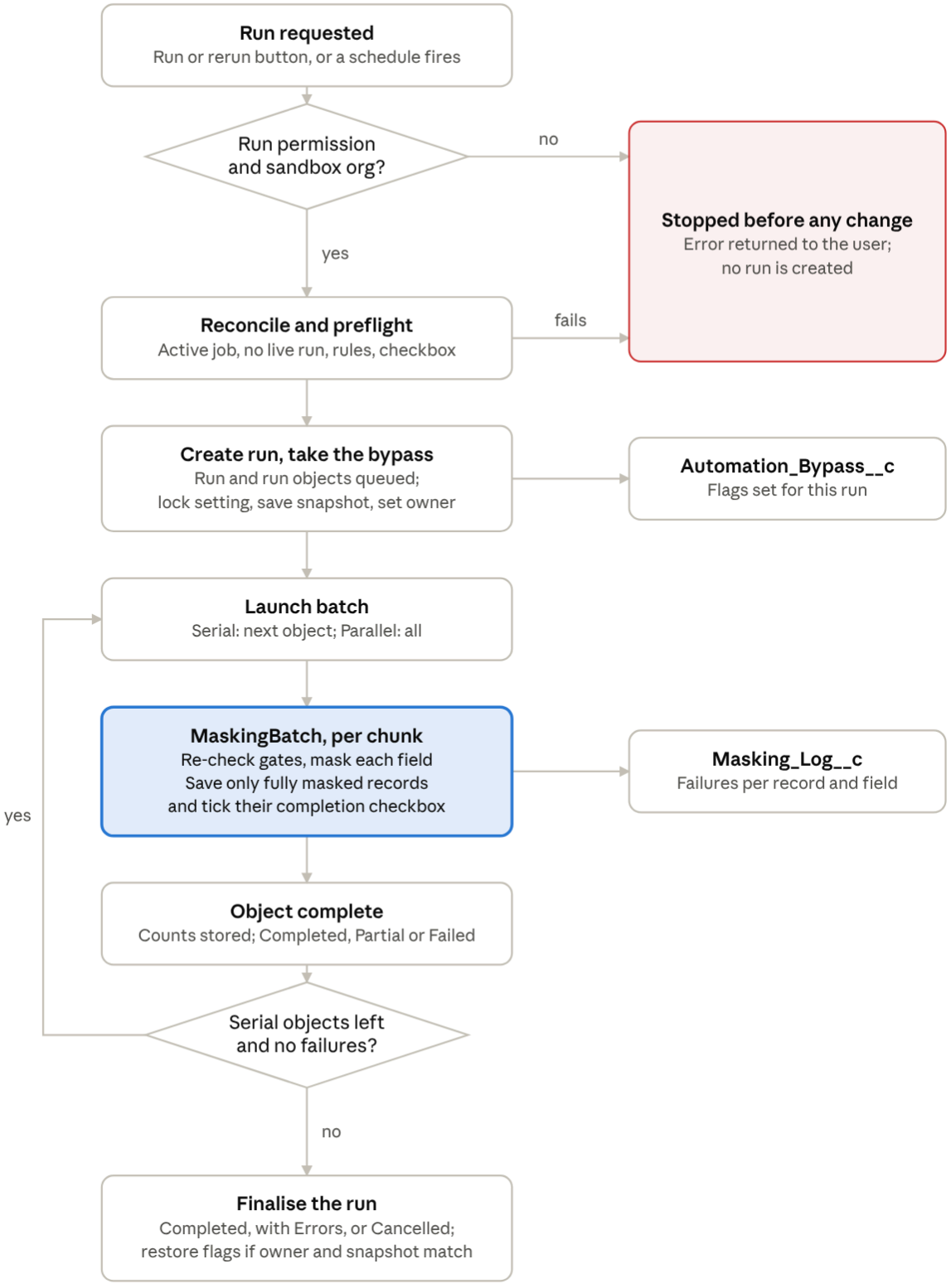
5.8 Audit Centre and dashboard

The Audit Centre filters activity by category (Data Masking, PII Discovery, Automation, Files, Configuration, Jobs), status, user, dates and text, and shows the detail of each entry. Configuration changes to field rules, jobs and job objects are captured automatically by Apex triggers. Secret values are redacted. The dashboard shows how many accepted fields are configured, how many objects and fields are active, and the latest run status.

6. Process flow: a masking run

A run is checked twice before it changes anything: at the request and inside every batch chunk. Automation bypass flags are held for the whole run and restored only by the run that set them.

A masking run restores automation only after every object finishes



Masking run lifecycle · 7 steps, 2 decisions

In a parallel job, every object launches at once and the run finalises when the last one finishes. A failed or partial object in a serial job marks the remaining queued objects Skipped.

Run and object statuses

Record	Statuses
`Masking_Job_Run__c`	Running → Completed, Completed with Errors, Cancelled or Failed
`Masking_Job_Run_Object__c`	Queued → Running → Completed, Partial, Failed, Skipped or Cancelled
`Deactivated_Automation__c` (trigger changes)	Queued → submitted and polled → verified at the target status, or Check required

Reconciliation: opening the control panel or starting a new run calls `RuntimeMaskingJobOrchestrator.reconcile`. It closes any run object whose batch ended without reaching `finish()`, for example after an exception in `start()`, and then finalises the run.

7. Technical component design

The 49 Apex classes fall into six layers. Controllers are thin; the logic sits in services. Every class is `public` (none is `global`), so subscribers can't call package Apex directly, apart from the one invocable action.

7.1 Apex classes

Layer	Class	Responsibility	Sharing
Controller	`PrivacyGuardHomeController`	Dashboard overview	with
Controller	`PIIScanReviewController`	Start scans, list findings, accept or remove	with
Controller	`PrivacyGuardConfigurationController`	Field rules and keywords, create/update/delete	with
Controller	`MaskingControlPanelController`	Facade for jobs, runs, preview and schedules	with
Controller	`AutomationManagementController`	Trigger discovery, deactivate, restore, re-check	with
Controller	`FileAttachmentPurgeController`	Object list and scan counts; purge requests are rejected	with
Controller	`PrivacyGuardFileInventoryController`	File inventory paging and selected deletion	with
Controller	`PrivacyGuardAuditController`	Audit list and detail	with
Invocable	`MaskingRunInvocable`	Flow action "Run Data Masking Jobs" (legacy metadata path)	with
Service	`MaskingJobManagerService`	Job CRUD, schedule, run, rerun, cancel, run details	with
Service	`RuntimeMaskingJobOrchestrator`	Start, launch, complete, reconcile and cancel job runs	with

Layer	Class	Responsibility	Sharing
Service	`MaskingJobOrchestrator`	Legacy metadata-driven orchestration	with
Service	`MaskingFrameworkService`	Legacy single entry point for a full metadata-driven run	with
Service	`MaskingConfigService`	Loads active field rules and the legacy job sequence	with
Service	`MaskingPreviewService`	Read-only preview, 5 records	with
Service	`MaskingCompletionService`	Resolves and validates the completion checkbox	with
Service	`MaskingConfigPromoter`	Generates custom metadata XML for accepted findings	with
Service	`PIIFieldScannerService`	Field detection rules	with
Service	`ComplianceCategorizationService`	Reads Data Classification from `FieldDefinition`	with
Service	`FileAttachmentPurgeService`	Retired object-wide purge entry point; always throws after the gates	with
Service	`AutomationManagerService`	Cooperative bypass: lock, snapshot, owner, restore	with
Service	`AutomationMetadataToggleService`	Retired Flow/VR entry points (always throw)	with
Service	`PrivacyGuardAuditService`	Configuration audit capture and activity records	**without**
Integration	`ToolingApiClient`	Same-org REST/Tooling calls with allowlists and org verification	with
Security	`PrivacyGuardSecurityService`	Custom permission checks; `hasProductionOverridePermission()` always returns false	with
Security	`EnvironmentSafetyService`	`IsSandbox` gate; fails closed	with
Security	`SchemaValidationUtil`, `PrivacyGuardSchema`	Schema-validated object and field names, type checks, value conversion	with
Async	`MaskingBatch`	Masks one object; enforces object and field edit access	**without**
Async	`MaskingFailedRecordRecoveryBatch`	Re-masks failed records from logs	**without**
Async	`FileAttachmentDeletionBatch`	Retired object-wide purge; fails closed in `start()` and `execute()`	with
Async	`PIIFieldScannerBatch`	Discovery across objects	with
Async	`AutomationTriggerToggleQueueable`	Trigger status deployment and polling	**without**
Async	`MaskingScheduler`, `PrivacyGuardWeeklyScheduler`, `MaskingSchedulerUtil`	Scheduled job execution and schedule management	with
Logging	`MaskingLogger`	Buffered and immediate log writes	with
Strategy	`IMaskingStrategy`, `MaskingStrategyFactory` + 10 strategy classes	Strategy pattern for field masking	with

7.2 Lightning Web Components

Component	Purpose
`privacyGuardHome`	Dashboard: coverage, objects, fields, privacy status
`piiScanReview`	Run New Scan, progress, candidate review by object
`piiKeywordConfiguration`	Keyword list: add, disable, remove
`dataMaskingConfiguration`	Field rules: strategy, pattern, active toggle, per-type sample values
`maskingControlPanel`	Job editor, run, rerun, cancel, schedule, preview, run details
`automationManagement`	Trigger list, deactivate, restore, deployment status
`attachmentFilePurge`	Per-object counts; purge button permanently disabled
`privacyGuardFileInventory`	File inventory, CSV export, selected deletion
`privacyGuardAuditCentre`	Activity explorer, filters, detail, export of loaded rows

Components call Apex imperatively and use only SLDS and base Lightning components. No external scripts or static resources are loaded.

7.3 Apex triggers

Trigger	Object	Events	Action
`PGAuditMaskingFieldConfig`	`Masking_Field_Config__c`	after insert, update, delete, undelete	Writes configuration audit entries
`PGAuditMaskingJob`	`Masking_Job__c`	after insert, update, delete, undelete	Writes job audit entries

Trigger	Object	Events	Action
`PGAuditMaskingJobObject`	`Masking_Job_Object__c`	after insert, update, delete, undelete	Writes job-object audit entries
`PGProtectMaskingLog`	`Masking_Log__c`	before insert, update, delete	Sanitises Failed entries; blocks every edit and delete

The three audit triggers are on the protected list in `PrivacyGuardAuditService.isProtectedTrigger`, so trigger management refuses to deactivate them. `PGProtectMaskingLog` isn't on that list, but in a subscriber org it is namespaced and therefore read-only in trigger management anyway.

8. Security design

Authorization relies on custom permissions checked in Apex, not on UI visibility. Destructive work also requires a sandbox org. Each check sits in the service or batch that does the work, so it can't be skipped by calling a lower layer.

8.1 Custom permissions

Custom permission	Gates	Checked in
`Manage_PrivacyGuard_PII`	Dashboard, discovery, keywords, field rules, file inventory (with Purge)	Controllers, `PIIFieldScannerService`
`Run_PrivacyGuard_Masking`	Jobs, preview, runs, reruns, schedules, bypass	Controllers, orchestrators, every masking batch `start()` and `execute()`
`Purge_PrivacyGuard_Files`	Selected file deletion	`PrivacyGuardFileInventoryController`; also checked before the retired purge entry points reject the request

Custom permission	Gates	Checked in
`Use_PrivacyGuard_Metadata_Deactivation`	Trigger discovery and changes	Controller and every `ToolingApiClient` call
`View_PrivacyGuard_Audit`	Audit Centre	`PrivacyGuardAuditController`
`Override_PrivacyGuard_Production_Block` (label: PrivacyGuard Retired Permission)	Nothing. Retired; no permission set grants it.	Not checked: `hasProductionOverridePermission()` returns false

8.2 Permission sets

Permission set	Grants	Intended for
PrivacyGuard Admin	Manage PII, Run Masking, Purge Files, View Audit; Apex class access; object CRUD with View All. `Masking_Log__c` is read-only (no create, edit or delete), and its core fields are read-only.	Package administrators
PrivacyGuard Operator	Run Masking; job controllers; edit on the completion checkboxes	Users who run masking jobs
PrivacyGuard Purge Operator	Purge Files; file controllers	Users who delete selected files
PrivacyGuard Tooling Access	External credential principal `PrivacyGuard_Tooling_OAuth-PrivacyGuard_User`	Trigger operators (authenticate individually)
PrivacyGuard Metadata Deactivation Break Glass	`Use_PrivacyGuard_Metadata_Deactivation`	Approved trigger administrators
PrivacyGuard Retired Control A (`PrivacyGuard_Production_Break_Glass`), PrivacyGuard Retired Control B (`PrivacyGuard_Production_Override`)	Nothing. Custom permission removed; descriptions say "Do not assign".	Retained for upgrade compatibility only

8.3 Sandbox gate

`EnvironmentSafetyService.assertExecutionAllowed()` reads `Organization.IsSandbox`. If the query fails, the org is treated as production. The check runs in every masking and recovery batch (`start()` and each `execute()`), in both orchestrators, in the scheduler, in every Tooling call and before selected file deletion. The retired production-override setting, custom permission and permission sets are not consulted. Discovery, configuration, preview, inventory and audit are not gated, because they don't change business data.

8.4 Sharing and field-level security

- **User-facing reads** use `with sharing` and `WITH USER_MODE`. That covers preview, inventory, scan counts, run and job queries, and trigger lists.
- **Selected file deletion** uses `Database.delete(..., AccessLevel.USER_MODE)`. It is now the only file-deletion path.
- **System context, by design (4 classes):**
  - `MaskingBatch` and `MaskingFailedRecordRecoveryBatch` bypass record sharing so masking reaches every record, including ones the running user can't see. Field-level security is enforced: before writing, `MaskingBatch` checks that the running user can edit the object, each target field and the completion checkbox. A field that fails the check is logged as a failure and its record isn't saved. The recovery batch reads failed records with `AccessLevel.SYSTEM_MODE` and then writes through the same `MaskingBatch` logic, so the same field-level checks apply.
  - `AutomationTriggerToggleQueueable` persists deployment state.
  - `PrivacyGuardAuditService` is insert-only audit capture.
- **Run bookkeeping writes** use `insert/update as system`.
- **Injection protection:** every dynamic object and field name is validated against the org schema (`SchemaValidationUtil.requireValidObject`, `requireValidField`, `requireSoqlIdentifier`) before it's put into SOQL. Tooling IDs are escaped with `String.escapeSingleQuotes`.

8.5 Authentication and credentials

Component	Configuration
Named Credential `privacyguard__PrivacyGuard_Tooling`	Subscriber sets its own sandbox My Domain URL. Generate Authorization Header on; merge fields off in header and body.
External Credential `PrivacyGuard_Tooling_OAuth`	OAuth, scope `api refresh_token`, per-user principal `PrivacyGuard_User`
External Auth Identity Provider `PrivacyGuard_Tooling_OAuth`	Subscriber enters the client ID and secret of their own OAuth app (PKCE on, exact callback)
Token storage	Salesforce-managed per-user tokens. No session ID used; no tokens in package data.

8.6 Data protection

- Business field values are never written to PrivacyGuard objects. Preview values exist only in the response to the user.
- **Failure logs are sanitised at write time.** Trigger `PGProtectMaskingLog` (before insert) replaces `Error_Message__c` on every `Failed` entry with a fixed message. It also clears `Technical_Details__c` and `Change_Details__c` and sets a generic `Admin_Message__c`. Record ID, object and field names are kept, so failed-record recovery still works.
- **The audit log is immutable.** The same trigger rejects every update and delete of `Masking_Log__c`, whoever the user, and the Admin permission set grants read access only.
- Configuration audit redacts secret values.
- All 9 PrivacyGuard custom objects have org-wide default **Private**. Administrators see all records through View All; other users see only records they own or that are shared with them.
- `Automation_Bypass__c` is a Public hierarchy setting so subscriber automation can read `$Setup` flags. Users with Customize Application can edit it.

9. Integration design: same-org Tooling API

PrivacyGuard has one integration: the subscriber org calling its own REST and Tooling API to change the status of an Apex trigger. Apex can't set a trigger's status directly, so the change is deployed through a Tooling `MetadataContainer`.

9.1 Trigger status change sequence

1. The operator selects eligible triggers and chooses Deactivate or Restore. The controller checks the permission, rejects namespaced and protected triggers, writes a `Deactivated_Automation__c` row (Queued, target status), and enqueues one `AutomationTriggerToggleQueueable` per trigger.
2. The queueable re-checks the permission and the sandbox gate. Then, once per transaction, it calls `GET /services/data/v66.0/query` for `Organization` and confirms the remote Org ID matches the local org and `IsSandbox` is true.
3. It reads the trigger (`Id, Name, Body, ApiVersion, NamespacePrefix`) and refuses if it is namespaced or protected.
4. It creates a `MetadataContainer`, then an `ApexTriggerMember` with the unchanged body and the target status, then a `ContainerAsyncRequest`.
5. It polls `ContainerAsyncRequest` up to 60 times, re-enqueueing itself with a 1-minute delay between polls and saving progress each time.

6. When the deployment completes, it reads `ApexTrigger.Status` and marks the change verified only if the actual status matches the target. A restore also sets `Reactivated__c`. Anything uncertain is left as Check required for the operator to re-check.

9.2 Interface specification

Call	Method	Resource	Allowlist / constraint
Org verification	GET	`/services/data/v66.0/query`	Fixed SOQL on `Organization`
Tooling query	GET	`/services/data/v66.0/tooling/query`	SOQL must be `SELECT … FROM ApexTrigger`, `ContainerAsyncRequest` or `ApexClass`
Tooling create	POST	`/services/data/v66.0/tooling/sobjects/{type}`	Type must be `MetadataContainer`, `ApexTriggerMember` or `ContainerAsyncRequest`; expects HTTP 201
Tooling update	PATCH	—	Always rejected

- **Transport:** HTTPS through `callout:privacyguard__PrivacyGuard_Tooling`. The authorization header comes from the user's own OAuth principal.
- **Timeouts:** 20 seconds per request.
- **Error messages:** errors surface as fixed messages that never include response bodies or credentials.
- **Diagnostics:** `diagnoseConnection()` returns only a pass or fail message.

10. Asynchronous processing, scheduling and limits

All data and metadata changes run asynchronously. Each async unit re-checks the permission and sandbox gates itself, so a queued job won't run if the org or the user's access changes after it was submitted.

Async unit	Type	Unit of work	Scale and limit notes
`PIIFieldScannerBatch`	Batch	1 object per `execute()`	Uses describe calls only; no record data

Async unit	Type	Unit of work	Scale and limit notes
`MaskingBatch`	Batch, Stateful	Records of one object per chunk	Batch size 1–2,000 (default 200). `Database.update(allOrNone = false)`. Logs buffered and flushed once per chunk
`MaskingFailedRecordRecoveryBatch`	Batch, Stateful	Failed log rows	Pages over log rows rather than holding all failed IDs in memory
`FileAttachmentDeletionBatch`	Batch (retired)	None	Fails closed in `start()`; no records queried or deleted
`AutomationTriggerToggleQueueable`	Queueable with callouts	One trigger change, one step	All callouts happen before DML in each run. Up to 60 polls, 1-minute delay
`PrivacyGuardWeeklyScheduler`, `MaskingScheduler`	Schedulable	One job, or the legacy sequence	Run as the user who scheduled them, with that user's permissions

Concurrency rules

- One active run per job. `start()` locks the job row (`FOR UPDATE`) and refuses if a Running run exists.
- One owner of the bypass at a time. The setting row is locked, and a second owner is refused.
- Run completion locks the run-object and run rows, so parallel `finish()` calls can't double-count or overwrite a final status.
- Parallel mode starts one batch per object at once. The number of objects in a parallel job counts against the org's limit of 5 concurrently processing batch jobs; additional batches wait in the flex queue.

Scheduling

Schedules are created with `System.schedule` in the scheduling user's time zone: daily, weekly on a chosen day, or monthly on day 1–28 or the last day. Cancelling a schedule aborts its scheduled jobs. Scheduled runs keep the sandbox gate, so a refresh into production can't run them.

Known limits in this release

Limit	Value
Preview	5 records, 100 fields per object

Limit	Value
Selected file deletion	100 items per request
Object count scan	10 objects per request
File inventory page	100 rows
Trigger discovery	50,000 triggers
Detection reason	255 characters
`LIST` and `NUMBER_RANGE` patterns	255 characters

11. Error handling, logging and audit

The design records every failure and keeps partial results safe. A record is never half-masked. A failed step stops the serial chain rather than continuing on bad state. Every operation leaves a log entry an administrator can find in the Audit Centre.

11.1 Error handling rules

Situation	Behaviour
Missing permission or production org	Typed exception before any work. The user sees a plain message. The invocable returns `success = false` with the message.
Invalid configuration (object, field, strategy, pattern, checkbox)	Rejected at save or at run preflight with a specific message. Nothing starts.
A field fails to mask, including when the running user can't edit the object, field or completion checkbox	The whole record is skipped and a Failed log entry is written per field. The record isn't marked complete.
A record fails to save	Partial save (`allOrNone = false`). A Failed log entry keeps the record ID, object and field. The Salesforce error text is replaced with a generic message when stored.
A batch ends without `finish()`	Reconciliation marks the run object and finalises the run
Serial object fails or is partial	Remaining objects are Skipped. Run ends Completed with Errors.

Situation	Behaviour
Bypass restore fails (wrong owner or bad snapshot)	Restoration is refused and state is kept for administrator recovery. The exception propagates so finalisation can be retried.
Tooling deployment slow or unclear	Status stays Queued or Check required. It is never marked failed without evidence.
Object-wide purge requested	`ObjectPurgeDisabledException`: "Object-wide purge is disabled. Use All files to review and explicitly select accessible items for deletion."
UI errors	Controllers throw `AuraHandledException` with user-safe text

11.2 Log model

`Masking_Log__c` holds three kinds of entry, separated by `Activity_Category__c` and `Status__c`:

- **Record failures:** run ID, object, record ID and field. The message is the fixed text "Operation failed. Review the operation status and affected record identifiers." Technical details are cleared.
- **Activity:** scans, runs, deletions, trigger operations, with an admin-facing message.
- **Configuration audit:** written by the three audit triggers for create, update, delete and undelete on field rules, jobs and job objects. `Change_Details__c` holds before and after values as JSON, with secret values redacted. It is cleared if the entry is a failure.

Logs are written in system mode, so the entry exists even when the acting user can't create log records. Log writes inside a batch chunk are buffered and flushed once. Trigger `PGProtectMaskingLog` runs before insert, update and delete: it sanitises Failed entries on insert and rejects every update and delete. This is covered by `PrivacyGuardLogProtectionTest`.

The trade-off is that the original Salesforce error text, such as a validation-rule message, is no longer kept anywhere. To diagnose a failure, administrators open the affected record by its logged ID and try the save themselves.

11.3 Retention

Log entries can't be edited or deleted by any user, including administrators, so there is no in-app or manual retention purge. Logs grow with every run until the sandbox is refreshed or the package is uninstalled.

12. Deployment, configuration and operations

The package installs with no post-install script and does nothing until an administrator grants access and configures it. Trigger management needs extra OAuth setup. Masking and discovery don't.

12.1 Installation and access

1. Install the package version 04tQE00000jpLenYAE in the target sandbox using **Install for Admins Only**.
2. Assign **PrivacyGuard Admin** to administrators. Assign **Operator** and **Purge Operator** only to the users who need them.
3. Open the **PrivacyGuard** Lightning app.
4. Create an editable `Data_Masking_Complete__c` checkbox, defaulting to false, on every object to be masked other than Account, Contact and Lead. Grant edit access on it to operators.

12.2 Trigger management setup (optional)

1. In the target sandbox, create an OAuth app with the exact callback URL shown on the installed **PrivacyGuard Tooling OAuth** provider. Turn on the `api` and `refresh_token` scopes and keep PKCE enabled.
2. Enter its client ID and secret in the installed External Auth Identity Provider, along with the sandbox's authorization and token URLs.
3. Set the installed Named Credential **PrivacyGuard Tooling** URL to the sandbox's own My Domain. Leave Allowed Namespaces blank.
4. Assign **PrivacyGuard Tooling Access** and **PrivacyGuard Metadata Deactivation Break Glass** to approved operators. Each operator then authenticates under their personal External Credentials settings.
5. Test by deactivating and restoring one eligible test trigger.

12.3 Operating procedure

When	Action
First use	Run PII discovery, review candidates, configure strategies, activate rules
Before a large run	Preview, run a small Full or Unmasked-only job, check run details and logs

When	Action
After each run	Check run status. Rerun failed records. Restore any triggers you deactivated.
After a sandbox refresh	Re-check completion checkboxes, schedules, the Named Credential URL and the OAuth callback. Re-authenticate operators.
Before an upgrade	Finish or cancel active runs, and review scheduled jobs
After an upgrade	Check permission set assignments and configuration, then run a small test
Regularly	Review permission assignments and access to the Audit Centre. Monitor data storage used by Masking_Log__c.

12.4 Subscriber automation

For subscriber Flows, validation rules and triggers to step aside during a run, they must check `$Setup.privacyguard__Automation_Bypass__c` flags (`Bypass_All_Automations__c`, `Bypass_Triggers__c`, `Bypass_Flows__c`, `Bypass_Validation_Rules__c`). Automation that doesn't check these flags runs normally during masking and may block saves. Those failures appear in the logs and can be fixed with failed-record recovery.

13. Testing approach

The package ships 47 Apex test classes and 2 test utility classes, with about 260 test methods. Tests use package-owned configuration rather than the shipped sample metadata, and grant the same packaged permissions a real operator would have.

Area	Test classes (examples)	What is proven
Authorization	`PrivacyGuardConsolidatedSecurityTest`, `PrivacyGuardInternalAuthorizationTest`, `MaskingJobEntrySecurityTest`, `MaskingJobCrudSecurityTest`, `PrivilegedBatchSecurityTest`, `AutomationTriggerSecurityTest`	Every entry point and privileged batch refuses users without the custom permission. Permission bypass is turned off in 8 classes and `System.runAs` is used in 10.

Area	Test classes (examples)	What is proven
Sandbox gate	`EnvironmentSafetyServiceTest`	Production is blocked, an unknown org type fails closed, and the retired override permission always returns false
Masking engine	`MaskingStrategyTest`, per-strategy tests, `RegexMaskSafetyTest`, `MaskingCompletionServiceTest`	Strategy output, pattern validation, completion checkbox rules, fail-closed record handling
Run lifecycle	`PrivacyGuardRuntimeLifecycleTest`, `MaskingFrameworkIntegrationTest`, `PrivacyGuardJobReportingTest`, `PrivacyGuardRunDashboardTest`	Serial and parallel runs, recovery, cancellation, reconciliation, counts
Bypass	`AutomationManagerServiceTest`, `AutomationBypassRestorationTest`	Single owner, snapshot restore, wrong-owner and invalid-snapshot rejection
Trigger management	`ToolingApiClientTest`, `AutomationTriggerProgressTest`, `PrivacyGuardTriggerBoundaryTest`	Allowlists, org verification, polling, protected and namespaced trigger refusal (HTTP mocks)
Discovery	`PIIFieldScannerTest`, `PIIClassificationDiscoveryTest`, `PIIScanReviewControllerTest`	Keyword token matching, Data Classification, review lifecycle
Files	`FileAttachmentDeletionTest`, `FileAttachmentPurgeControllerTest`, `PrivacyGuardFileInventoryTest`	Limits, confirmation, user-mode deletion; object-wide purge fails closed
Scheduling	`MaskingSchedulerTest`, `MaskingSchedulerUtilTest`, `MaskingFrequencyScheduleTest`	Daily, weekly and monthly expressions; cancellation
Audit and config	`PrivacyGuardAuditTest`, `PrivacyGuardConfigLifecycleTest`, `MaskingLoggerTest`	Audit capture, redaction, log persistence; `PrivacyGuardLogProtectionTest` proves failure text is suppressed and log edits and deletes are rejected

Acceptance testing in a subscriber sandbox

1. Access boundaries: a privileged user and a user with no PrivacyGuard permissions.
2. Discovery and configuration with synthetic data.
3. A small Full job, then an Unmasked-only job, then failed-record recovery.
4. Cancellation of a multi-chunk run, and bypass restoration.
5. Deactivation and restore of a test trigger, plus wrong-org and namespaced refusals.
6. Selected deletion on disposable files, and confirmation that object-wide purge is refused.

7. In a production or Developer Edition org, refusal of masking, purge and trigger changes.

Code coverage figures and static-analysis results for build 0.2.0.3 are kept with the security review evidence. They aren't repeated here.

14. Constraints

The design's main constraints follow from what can't be undone: masking and deletion are permanent.

14.1 Constraints

- Masking and deletion can't be reversed. Recovery means refreshing the sandbox.
- Cancelling stops only future chunks. Committed chunks stay masked.
- Completion is tracked by checkbox. Values put back into masked records later by users or integrations aren't detected unless the checkbox is reset.
- Cooperative bypass works only for automation written to check the flags. Flows and validation rules can't be deactivated by the package.
- Custom strategy classes must live in the package, because `IMaskingStrategy` isn't `global`.
- `EMAIL` keeps the original domain. Use `EMAIL_INVALID` where the domain itself is sensitive.
- Discovery uses metadata signals only. It can't prove that every PII field has been found, and it never inspects file contents.
- File deletion is item-by-item only (up to 100 per request, within the user's access). Clearing all files from an object needs repeated selected deletions or a sandbox refresh.
- Failure logs keep identifiers but not the Salesforce error text.
- Audit log entries can't be deleted, so they count against data storage until the sandbox is refreshed.
- Masking only covers fields the running user can edit. Operators, and the users who own schedules, need edit access to every configured field and to the completion checkbox on each masked object. Otherwise those records fail and stay unmasked.